LOG6951A-TP1: ResearchPal

Felix Paille Dowell,
2256243

POLYTECHNIQUE
MONTRÉAL

Introduction:

ResearchPal est une application qui a pour objectif d'aider à interroger une base de données à l'aide d'LLM. Il se fait avec une implémentation de *Retrieval-Augmented Generation*, ou RAG. Le corpus peut inclure toute source que l'utilisateur souhaite inclure. Pour le but de ce TP, le corpus sera composé d'articles sur les groupes de rock ayant des grandes influences: Led Zeppelin, Nirvana, Black Sabbath, etc. On suppose alors un utilisateur amateur de ce genre de musique qui souhaite faire des recherches sur le sujet. Ce sujet à été choisi car il y a plusieurs sources en ligne qui répètent les mêmes informations, comme les noms des membres ou les albums qui sont sortis, ce qui permet de tester la performance de notre méthode de retrieval.

Architecture:

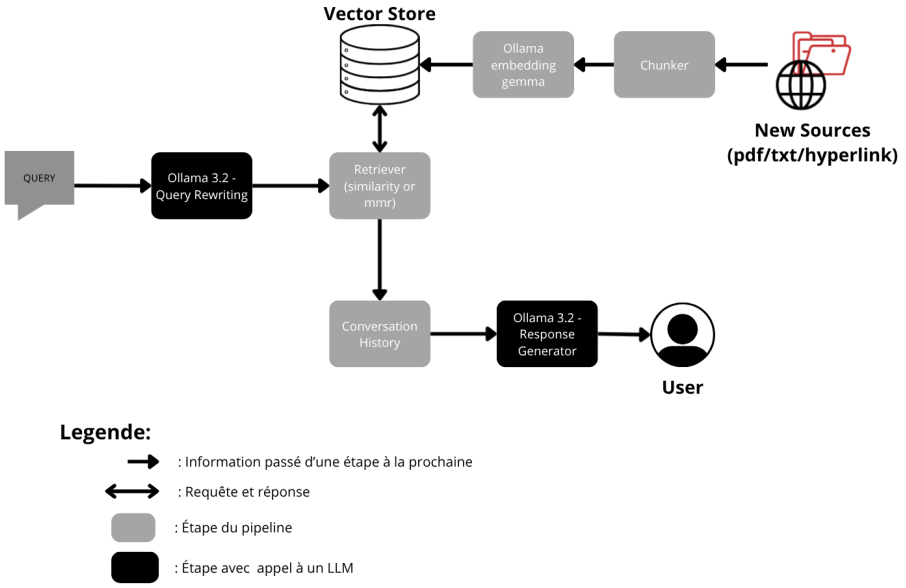

Figure 1 - Diagramme de l'architecture de l'application ResearchPal

Choix technologiques:

Le LLM choisi est Llama 3.2 via Ollama. Ce choix s'explique par le fait que les LLMs accessibles via des API gratuites en ligne imposent souvent des limites d'utilisation, ce qui complique le développement et les expérimentations. Les modèles Ollama sont locaux et donc n'ont pas de limites d'utilisation. LLama 3.2 est un modèle de taille et de complexité moyenne, ce qui est parfait pour notre application. L'inconvénient est qu'il faut télécharger le modèle sur sa machine pour pouvoir utiliser l'application.

Le choix du modèle d'embeddings découle du choix d'LLM. Nous utilisons le modèle embedding gemma de Ollama. Il est un modèle de taille et complexité moyenne, ce qui est bon pour ce TP. Ce choix simplifie l'application car nous pouvons utiliser Ollama pour les embeddings ainsi que la génération.

Comme outil d'observabilité, j'ai choisi LangSmith. C'est l'option la plus simple, car l'application utilise LangChain.

Patron de prompt:

System prompt:

You are ResearchPal, a factual research assistant. Your role is to help the user answer questions based strictly on the provided sources.

You must follow these rules:

1. Answer only from the provided context.
2. Be transparent when the context is insufficient.
3. Never cite using bare numbers such as [1] or [2].
4. Every citation must include the full source metadata exactly as provided in the context, including source, title, and page when available.
5. End each answer with a **"Sources used:"** section listing the full references you relied on.

User prompt template:

Question: `{question}`
Conversation history: `{conversation_history}`
Context: `{context}`

Write a concise answer grounded only in the provided context, and include citations.

Figure 2 - Prompts pour l'étape de génération de réponse

Notre implémentation combine deux patrons de prompt complémentaires : Persona et Recipe. D'une part, nous attribuons explicitement au modèle le rôle de *ResearchPal*, un assistant de recherche factuel, afin d'encadrer son comportement et son ton de réponse. D'autre part, nous utilisons un patron de type Recipe en définissant une série d'instructions explicites et structurées (répondre uniquement à partir du contexte, signaler les limites de l'information disponible, citer les sources).

Cette combinaison permet de guider le modèle à la fois sur ce qu'il est (son rôle) et sur comment il doit répondre (ses contraintes), ce qui améliore la cohérence des réponses générées.

Comparaison des stratégies de retrieval:

Tableau 1 - Résumés des chunks obtenus sur différentes requêtes pour sim et mmr

Requête	Similarité (résumé des résultats)	MMR (résumé des résultats)
Qui étaient les membres de Led Zeppelin ?	Résultats dominés par Britannica et Wikipedia ; informations pertinentes mais redondantes, avec répétition des mêmes sources.	Résultats plus diversifiés (3 sources distinctes) ; conserve les informations clés tout en ajoutant une source supplémentaire, réduisant la redondance.
Quels sont les groupes de rock les plus influents de tous les temps ?	Résultats variés mais parfois incohérents (Pink Floyd, The Who, Rolling Stones), avec duplication de certaines sources.	Diversité maximale (4 sources uniques) incluant plusieurs groupes majeurs (Pink Floyd, Beatles, Led Zeppelin), offrant une vision plus large et représentative.
Quels groupes du corpus sont associés au heavy metal ?	Résultats concentrés sur Deep Purple et Metallica, avec répétition de la même source (Deep Purple).	Meilleure diversité : introduction de Black Sabbath en plus de Deep Purple et Metallica, donnant une couverture plus complète du genre.
Quel artiste est connu comme le chanteur principal de Queen ?	Résultats pertinents mais indirects (Queen, discographie) ; l'information sur Freddie Mercury n'est pas immédiatement centrale.	Résultats plus ciblés : inclut directement Freddie Mercury, ce qui améliore la précision tout en conservant plusieurs sources.
Quels groupes du corpus sont liés au grunge ou au rock alternatif ?	Résultats centrés sur Nirvana et Soundgarden, avec duplication des sources.	Diversité accrue (4 sources uniques) incluant Nirvana, Soundgarden, Pearl Jam ; meilleure couverture du mouvement grunge/alternatif.

* Ce tableau est généré par ChatGPT à partir de la sortie du script retrieval_method_compare.py

La recherche par similarité retourne généralement les documents les plus proches en termes de score, mais tend à produire des résultats redondants provenant des mêmes sources. À l'inverse, la méthode MMR favorise la diversité des sources, comme observé par le nombre plus élevé de sources uniques dans plusieurs requêtes. Cela permet d'obtenir une vision plus complète du sujet, bien que cette diversification puisse parfois introduire des résultats légèrement moins pertinents individuellement.

Au-delà des tendances générales, certaines différences concrètes illustrent bien les effets des deux approches. Par exemple, pour la requête sur le heavy metal, la similarité retourne surtout Deep Purple et Metallica, avec redondance, tandis que MMR introduit Black Sabbath, un groupe clé absent auparavant, améliorant la pertinence globale. De même, pour le grunge, MMR ajoute Pearl Jam en plus de Nirvana et Soundgarden, enrichissant la couverture du sujet. Pour les groupes influents, MMR introduit aussi The Beatles, absents en similarité.

En pratique, MMR est plus adapté aux requêtes exploratoires, alors que la similarité reste efficace pour des questions précises.

Les paramètres de retrieval ont été choisis afin de faciliter une comparaison claire entre les méthodes. Le paramètre $k = 4$ permet d'obtenir un nombre limité de résultats, suffisant pour observer les phénomènes de redondance et de diversité tout en restant analysable manuellement. Le paramètre $fetch_k = 10$ offre à MMR un ensemble de candidats plus large, ce qui améliore sa capacité à sélectionner des documents diversifiés. Enfin, $\lambda = 0.5$ établit un équilibre entre pertinence et diversité, permettant d'évaluer de manière neutre le compromis entre ces deux objectifs.

Les résultats d'exécution sont sur dans retrieval_method.txt, compris dans ResearchPal/src/test_scripts/outputs.

Technique d'optimisation de la requête:

Tableau 2 - Résultats de test de la méthode d'amélioration de requêtes par réécriture

Requête initiale	Sans query rewriting (résumé)	Requête réécrite	Avec query rewriting (résumé)
"the drummer guy?"	Réponse vague : liste plusieurs batteurs sans identifier clairement une personne précise ; forte ambiguïté.	**"Who is the prominent drummer in rock music?"**	Réponse mieux structurée et plus générale sur des batteurs importants, mais reste non spécifique à un individu précis.
"lead zeplin best stuff"	Réponse peu pertinente : discussion générale sur l'influence et les controverses, sans exemples concrets.	**"Top rock songs and albums by Led Zeppelin"**	Réponse beaucoup plus pertinente avec exemples précis (*Stairway to Heaven*, *Whole Lotta Love*).
"that band from seattle with kurt"	Réponse correcte mais indirecte : Nirvana est identifié avec du contexte inutile.	**"Band from Seattle featuring Kurt Cobain"**	Réponse plus directe et claire : identification explicite de Nirvana avec justification structurée.

* Ce tableau est généré par ChatGPT à partir de la sortie du script query_rewriting_compare.py

Les requêtes des utilisateurs peuvent parfois être vagues, ambiguës ou contenir des erreurs d'orthographe ou de syntaxe. Elles peuvent également manquer de mots-clés pertinents, ce qui nuit à la qualité du retrieval. Afin de pallier ces limitations, notre application intègre une étape de query rewriting, qui consiste à reformuler les questions de l'utilisateur en requêtes plus claires, structurées et riches en information.

Cette reformulation est effectuée à l'aide d'un LLM guidé par un system prompt spécialisé, conçu en fonction du corpus utilisé. Dans notre cas, le prompt précise que les documents portent sur le domaine du rock and roll, ce qui permet d'orienter la reformulation vers des termes plus pertinents pour ce contexte.

Ainsi, le query rewriting améliore la qualité des requêtes soumises au module de retrieval, ce qui se traduit par des résultats plus pertinents et mieux adaptés à l'intention de l'utilisateur.

```
You rewrite user questions into clearer retrieval queries for a RAG system
focused on rock and roll music.
Expand ambiguous references, preserve the original intent, and add useful
specificity when needed.
Return only the rewritten query.
```

Figure 3 - System prompt pour l'étape de query rewriting

Le query rewriting permet de transformer des requêtes ambiguës ou mal formulées en requêtes plus explicites, ce qui améliore significativement la pertinence des réponses. Par exemple, la reformulation de "lead zeplin best stuff" permet d'obtenir des résultats concrets (chansons spécifiques), alors que la requête initiale menait à une réponse vague. Toutefois, lorsque l'intention reste trop floue (ex. "the drummer guy"), la reformulation améliore la structure de la réponse sans garantir une identification précise.

Utilisation d'IA générative:

Codex et ChatGPT ont été utilisés afin de concevoir l'application. La quasi-totalité du code est écrit par codex. Je donnais des instructions sur les choix architecturaux (implémenter mmr, implémenter la query rewriting) et codex générait le code.

ChatGPT a été utilisé pour aider à rédiger ce rapport. Il était utilisé pour générer les tableaux (1, 2) et pour correction d'orthographe et reformulation de phrases.

Conclusion:

Malgré les résultats satisfaisants obtenus, plusieurs limites ont été observées dans le pipeline RAG. Tout d'abord, la qualité des réponses dépend fortement du retrieval: si les documents récupérés sont incomplets ou peu pertinents, le LLM ne peut pas produire une réponse adéquate. De plus, certaines requêtes ambiguës restent difficiles à interpréter, même avec le query rewriting. J'ai également constaté que la gestion du contexte conversationnel reste limitée, notamment pour des interactions longues ou complexes. Enfin, le système ne dispose pas de mécanisme de validation des réponses, ce qui peut mener à des réponses incomplètes ou imprécises.

Dans le cadre du TP2, qui introduit la programmation agentique, plusieurs pistes d'amélioration peuvent être envisagées. L'utilisation d'agents permettrait notamment de décomposer les requêtes complexes en sous-tâches, d'effectuer plusieurs étapes de retrieval ou de raisonnement, et de valider les réponses avant de les retourner à l'utilisateur. Il serait également possible d'intégrer des outils externes (par exemple pour effectuer des recherches supplémentaires ou filtrer les résultats), ainsi qu'un mécanisme d'auto-évaluation ou de critique pour améliorer la fiabilité des réponses. Enfin, un agent pourrait adapter dynamiquement les stratégies de retrieval ou de prompting en fonction du type de requête.